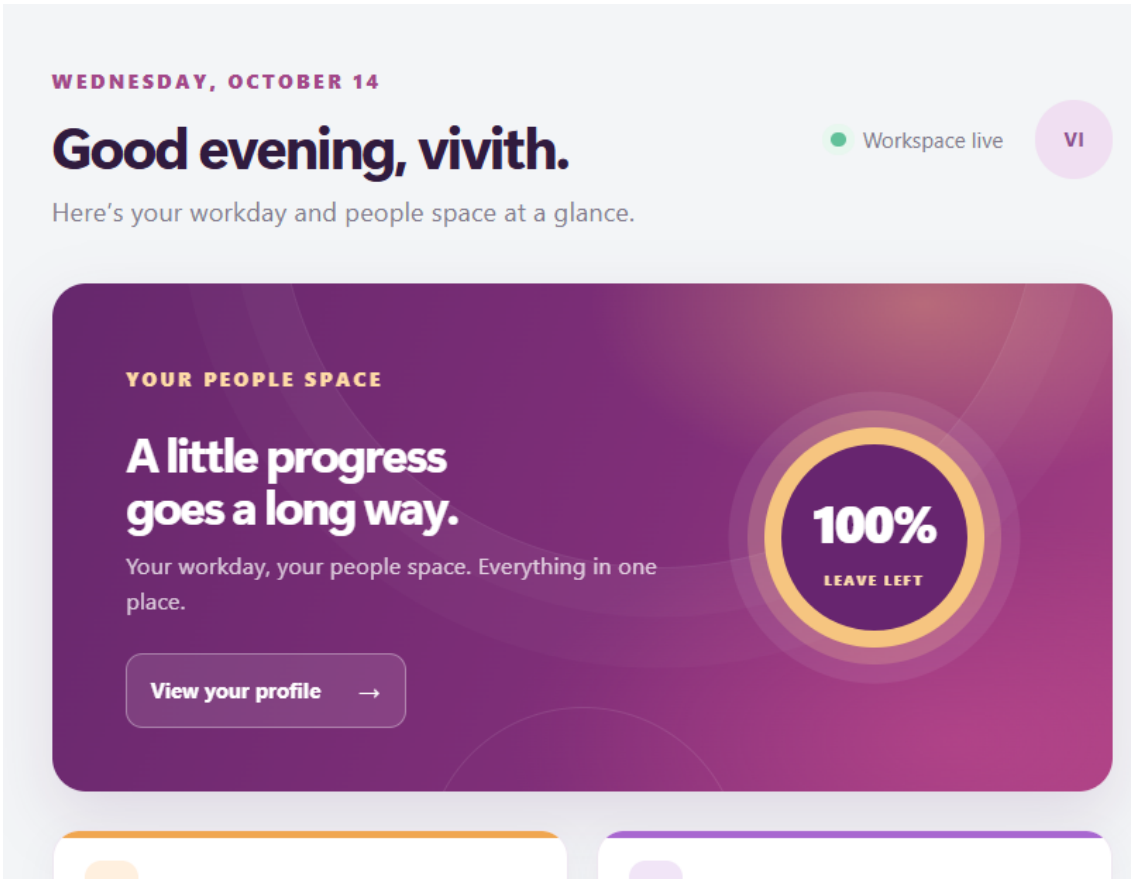
Human Resource Management System

Project report · local application walkthrough · feature, API, and test evidence

One role-aware workspace for people operations
Employee onboarding, attendance, leave, payroll, salary slips, reports, notifications, and audit history.



Output screenshot 1 — dashboard captured from the locally running app. This browser session is signed in as Employee; values are local demo data.

Frontend	Backend	Validation
React · Vite	FastAPI · SQLAlchemy	16 tests · build PASS

1. Scope and system architecture

The HRMS provides separate experiences for employees and people-operations staff. The dashboard links to focused modules rather than placing every task on one screen. Backend permissions remain the authority for protected actions.

Implemented modules

Area	Primary capability
Identity	JWT sign-in, role-aware navigation, account and role controls.
People	Employee onboarding, profile fields, directory, document upload and access.
Work time	Check-in/out, history, monthly totals, leave requests and decisions.
Compensation	Payroll generation, deductions/adjustments, PDF salary slips and exports.
Operations	Notifications, audit events, dashboards and filtered reports.

Request path

Browser	API	Persistence
React routes and forms call the API using the signed-in session.	FastAPI validates inputs, JWT identity, role permissions, and workflow rules.	SQLAlchemy persists records; local development defaults to SQLite, with MySQL supported.

Stack: React 18, React Router, Vite, Bootstrap, Axios, FastAPI, SQLAlchemy, Pydantic settings, JWT, SQLite for local development, and MySQL 8+ through PyMySQL.

This report documents the local application only. It does not claim that the system has been deployed to a public or production environment.

2. Authentication and role permissions

Users authenticate through the login endpoint and receive a signed bearer token. Protected API routes resolve the current user and check role and ownership before allowing access. Frontend route hiding improves usability but is not the security boundary.

Role	Typical access	Restricted actions
Admin	Organization-wide users, employee records, payroll, reports, settings, audit logs.	Sensitive changes are permission-checked and recorded.
HR	Employee operations, attendance review, leave decisions, payroll and reports as authorized.	Admin-only role/configuration and audit surfaces remain restricted.
Employee	Own profile, documents, attendance, leave, pay history, and personal reports.	Cannot approve/reject leave or access another employee's private records.

Security controls

• Passwords are stored as hashes; JWT signing uses a configured secret.
• Protected API operations require bearer authentication; the live OpenAPI contract lists the protected routes.
• Employee document access is authenticated and ownership-scoped; accepted document formats and size are restricted by the app.
• Leave decisions, sensitive payroll actions, and administrative events are permission-gated and covered by workflow tests.

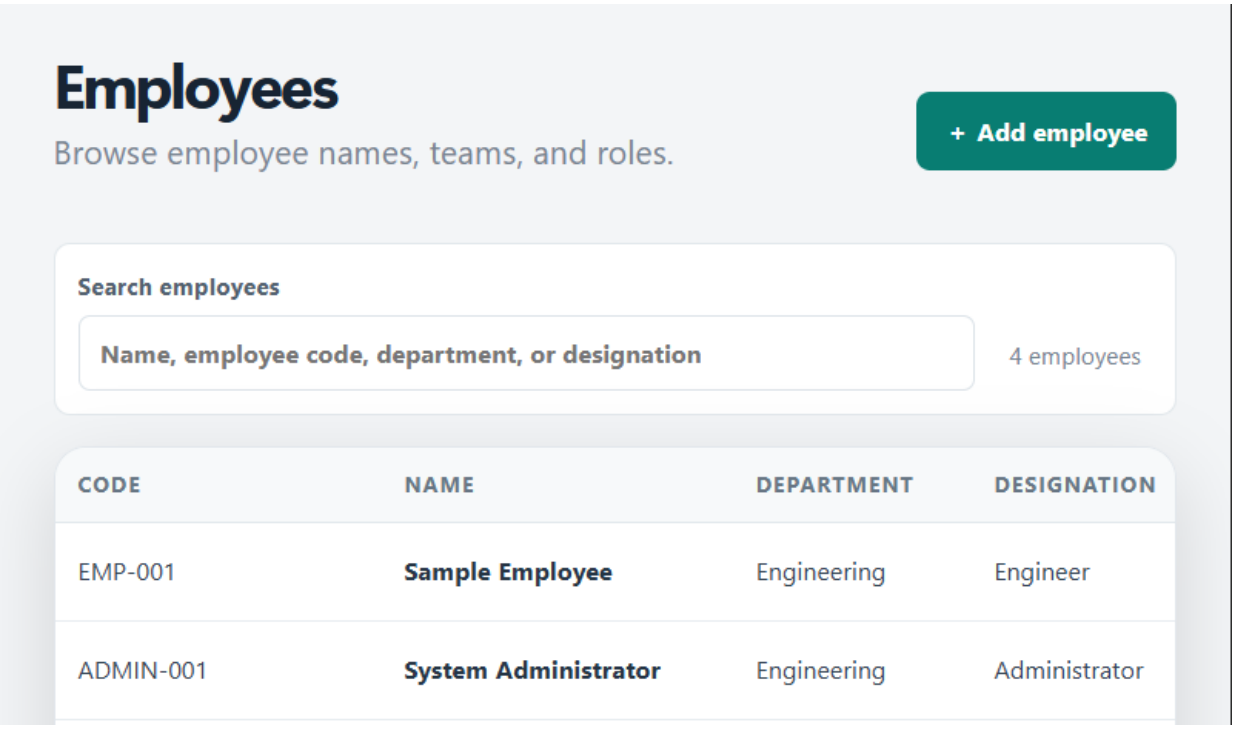
Configure a unique JWT secret and local passwords before use. The report and Postman collection intentionally contain no active passwords or access tokens.

3. Employee management

Employee Management supports a searchable directory and onboarding/profile workflows. Authorized HR/Admin users manage full employee records; employee accounts are restricted to their own private profile and documents.

Workflow

1. HR/Admin opens the employee directory and creates an employee account with work and department information.
2. Profile details can include contact, address, emergency contact, designation, employment status, and compensation fields.
3. The employee signs in to review their own profile and upload permitted PDF, JPEG, or PNG documents (up to 5 MB).
4. Document access and employee changes are authenticated; administrative activity is available in audit history.

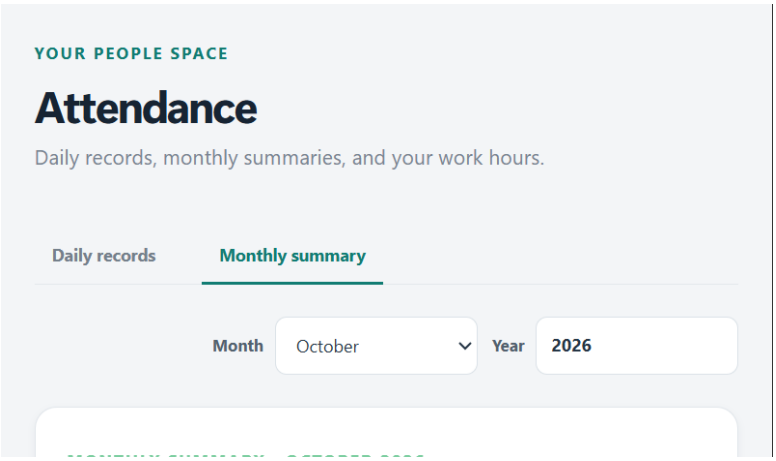


Output screenshot 2 — employee directory view from the current local Employee session. The full management actions remain HR/Admin-only.

4. Attendance and leave workflows

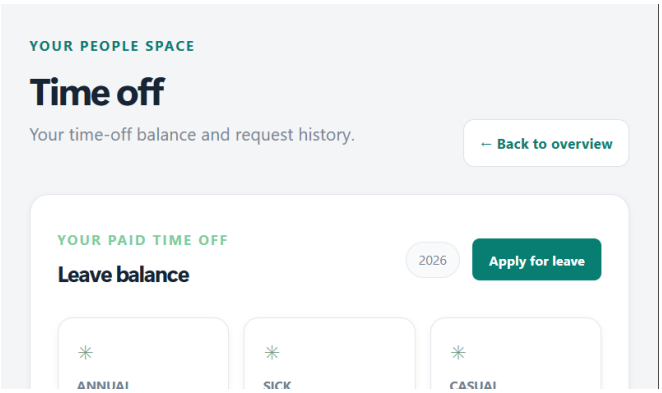
Attendance

Employees check in and check out from the attendance page. The app rejects duplicate transitions, presents daily history, and summarizes monthly working days and hours. HR/Admin can review authorized team data; privileged corrections are audited.



Leave

Employees submit a leave type, date range, and reason, then follow its pending, approved, or rejected state and balance. HR/Admin reviews the queue and records the decision. Approved leave affects the available balance and payroll loss-of-pay calculation where applicable.

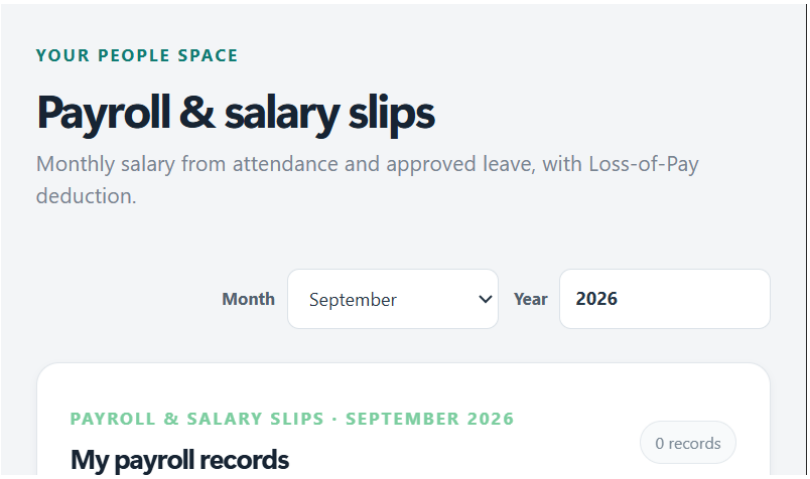


Output screenshots 3–4 — attendance and leave pages. These are employee-view captures; they do not show the HR/Admin approval queue.

5. Payroll and PDF salary slips

Payroll generation calculates period earnings and deductions from configured salary data, attendance, approved leave, loss-of-pay, and recorded adjustments. Payroll history is available to the employee for their own records and to authorized HR/Admin users for team processing.

Slip section	Included information
Identity and period	Employee/payroll reference and the salary month.
Earnings/deductions	Itemized salary components, deductions, LOP, and applicable adjustments.
Net payment	Net amount, amount in words, payment/simulation reference when applicable.
Privacy	Confidentiality footer; PDF download requires the relevant authorization.

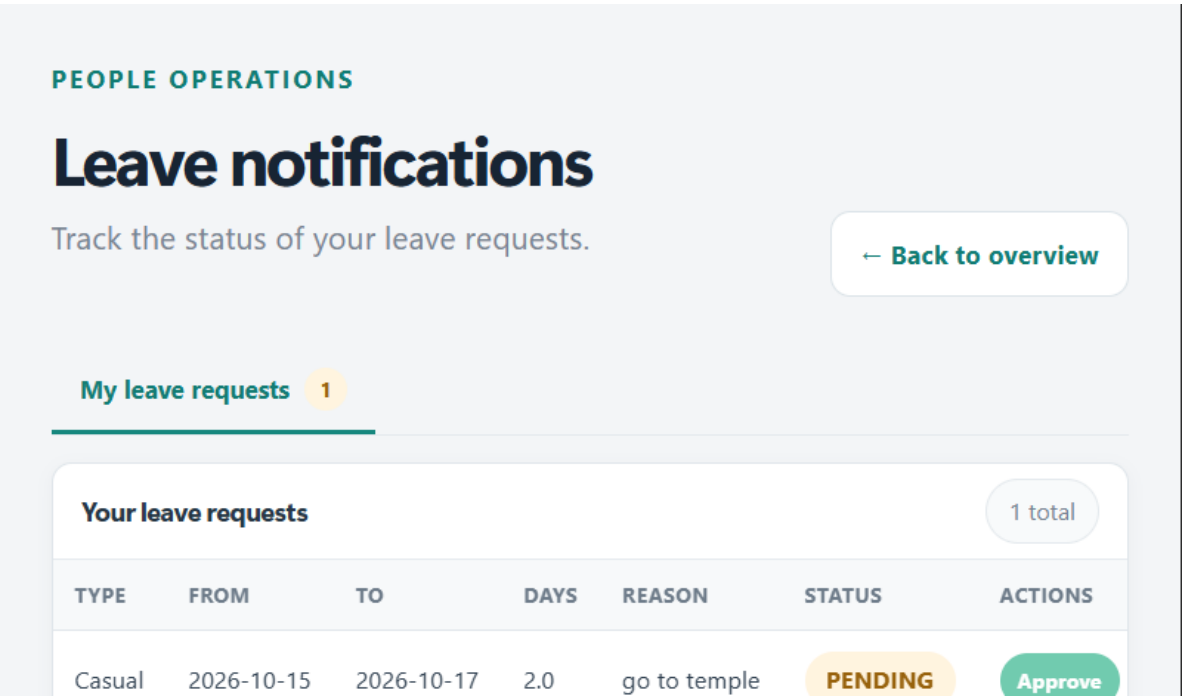


Output screenshot 5 — employee payroll page. The selected month has no generated records in this session; the fictional sample statement is provided separately.

Sample statement: deliverables/sample-salary-slip.pdf (fictional demo record; not a real payroll statement).

6. Notifications and audit history

The Notifications page presents leave request status to employees and workflow notifications/logs to authorized HR/Admin staff. Approve and Reject actions belong to the HR/Admin review queue; they are not employee self-service actions.



Output screenshot 6 — employee-facing leave status view from the local browser. The visible Approve/Reject buttons are disabled for this Employee account. The HR/Admin review queue and Admin-only audit records were not captured in this session.

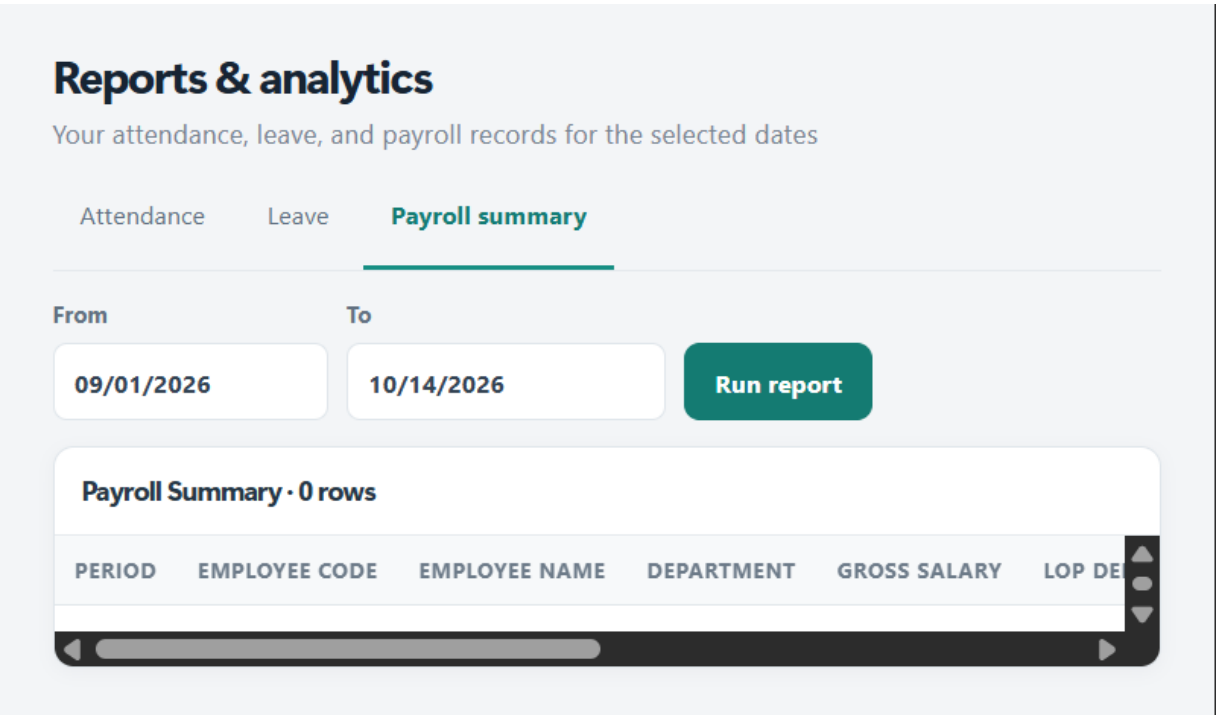
Audit trail

Administrative activity records identify the actor, action, affected entity, and time. Audit history is Admin-only. Email delivery outcomes can be reviewed in the authorized notification logs; SMTP must be configured for actual email delivery.

Verification includes allow/deny role checks for leave decisions and checks that decisions result in the expected request status, notification, and audit event.

7. Reports and analytics

Reports & Analytics groups Attendance, Leave, and Payroll summary views with date filtering and report/export actions. Employees receive personal data only; company-wide report access is limited to authorized roles.



Output screenshot 7 — local Reports & Analytics page, captured in Employee context. The visible payroll summary is empty for the selected range; it is not an Admin/HR organization-wide report.

Report paths

The API provides date-filtered report data and downloadable CSV/PDF outputs for authorized users. Report scope is enforced server-side. Filters support selected dates and, where authorized, department-level review.

8. API integration and verification

The frontend integrates with the FastAPI backend for authentication, dashboards, employee data, attendance, leave, payroll, reports, notifications, and audit operations. OpenAPI provides the machine-readable contract and Swagger/ReDoc provide local interactive documentation.

Check	Latest local result
Backend tests	16 passed, 1 warning in 12.84s
Frontend production build	✓ built in 3.97s
Health endpoint	ok (HTTP 200 docs UI)
API contract	40 paths; 48 operations; 43 bearer-protected operations.

Quality notes

The complete local check runs backend workflow tests, the frontend production build, and live health/docs/OpenAPI checks. It uses a disposable SQLite database and fictional test records; it does not alter the configured HRMS database. The latest run completed with an overall PASS.

Run timestamp: 2026-10-14T18:44:53.9024252+05:30. One warning is reported by the test runner; it did not fail the suite.

9. Local setup and handover

Start locally (Windows PowerShell)

Use the existing local Python environment and configure a unique JWT secret and Admin password. Keep `.env` and passwords out of source control.

```
cd backend
.\.venv\Scripts\Activate.ps1
Copy-Item .env.example .env
# Set a unique JWT_SECRET_KEY and local Admin credentials
python -m app.seed
uvicorn app.main:app --reload
```

In a second terminal:

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5173; API docs are available at http://localhost:8000/docs. For repeatable verification, run `backend\run_live_checks.ps1` from PowerShell inside the backend directory.

Project deliverables

- Database schema: database/hrms-schema.sql; setup notes: database/schema-notes.sql.
- API collection: postman/HRMS.postman_collection.json. Set local environment variables for credentials before running it.
- Sample salary slip: deliverables/sample-salary-slip.pdf; fictional data only.
- Safe credential provisioning: deliverables/sample-test-credentials.md (no active passwords).
- Test report and machine-readable contract: deliverables/live-test-results.json and deliverables/live-openapi.json.
- UI evidence: deliverables/hrms-project-report-assets/.

Admin/HR-only approval/audit views require signing in locally with an appropriately provisioned role. This report uses Employee-session screenshots and does not include credentials or claim a public deployment.